Exception Handling Rules for FreeST

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

Bruno Faustino
fc59784

2026/2027

Contents

1 Syntax **2**

2 Typing Rules **2**

3 Reduction Rules **2**

4 Some Thoughts about Shared Channel Cancellation **3**

4.1 Counter 3

4.2 Register First 4

4.3 Store Shared Channels 4

4.4 Increase Counter on send_ instead of fork 4

4.5 Is cancelling necessary for shared channels anyway? 4

1 Syntax

Multiplicity	$m ::= \mathsf{un} \mid \mathsf{lin}$
Basic Kinds	$\upsilon ::= \mathbf{C} \mid \mathbf{S} \mid \mathbf{T}$
Kinds	$\kappa ::= \upsilon^m$
Polarity	$\sharp ::= m! \mid m?$
View	$\star ::= m\oplus \mid m\&$
Channel End	$\mathsf{End} ::= \mathsf{Close} \mid \mathsf{Wait}$
Types	$T, U ::= \mathsf{Skip} \mid \sharp T \mid \star\{\ell\colon T_\ell\}_{\ell\in \mathrm{L}} \mid \mathsf{End} \mid T;T$
	$()_m \mid T \times T \mid T \to_m T \mid \{\ell\colon T_\ell\}_{\ell\in \mathrm{L}} \mid \langle \ell\colon T_\ell\rangle_{\ell\in \mathrm{L}}$
	$\forall a\colon \kappa\ .\ T \mid \mu a\colon \kappa.T \mid a$
Kinding contexts	$\Delta ::= \varepsilon \mid \Delta, a\colon \kappa$
Variables	x, y
Terms	$L, M, N ::= x \mid \lambda_m x\colon T.M \mid M\ N \mid ()_m \mid (M, N)$
	$\mathbf{let}\ ()_m = M\ \mathbf{in}\ N \mid \mathbf{let}\ (x, y) = M\ \mathbf{in}\ N$
	$\mathbf{channel}\ @T \mid \mathbf{fork}\ M \mid \mathbf{send}_m\ M\ N \mid \mathbf{receive}_m\ M$
	$\mathbf{sendType}\ @T\ M \mid \mathbf{receiveType}\ M$
	$\mathbf{select}_m\ \ell \mid \mathbf{case}\ M\ \mathbf{of}\ \{m\&\ell_i\ M_i \to N_i\}^{i\in 1..n}$
	$\mathbf{close}\ M \mid \mathbf{wait}\ M \mid \mathbf{cancel}\ M \mid \mathbf{raise}$
	$\mathbf{try}\ L\ \mathbf{as}\ x\ \mathbf{in}\ M\ \mathbf{otherwise}\ N$
Typing contexts	$\Gamma ::= \varepsilon \mid \Delta, x\colon T$
Configurations	$\mathcal{C}, \mathcal{D}, \mathcal{E} ::= (va)\mathcal{C} \mid \mathcal{C} \parallel \mathcal{D} \mid \lightning a \mid a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W})$
Evaluation contexts	$\mathcal{F} ::= \ldots$
Pure contexts	$P ::= \ldots$
Configuration contexts	$\mathcal{G} ::= [\,] \mid (va)\mathcal{G} \mid \mathcal{G} \parallel \mathcal{C}$

2 Typing Rules

$$\frac{\Gamma \vdash M\colon T \qquad \Delta \vdash T\colon \mathbf{S}^{\mathsf{lin}}}{\Gamma \vdash \mathbf{cancel}\ M\colon ()}\ \textsc{T-CANCEL} \qquad \frac{}{\vdash \mathbf{raise}\colon T}\ \textsc{T-RAISE} \qquad \frac{\Gamma_1 \vdash L\colon T_1 \qquad \Gamma_2, x\colon T_1 \vdash M\colon T_2 \qquad \Gamma_2 \vdash N\colon T_2}{\Gamma_1, \Gamma_2 \vdash \mathbf{try}\ L\ \mathbf{as}\ x\ \mathbf{in}\ M\ \mathbf{otherwise}\ N\colon T_2}\ \textsc{T-TRYIN}$$

3 Reduction Rules

Term Reduction $\boxed{M \to_M N}$

$$\textsc{E-TRYINVAL} \qquad \mathbf{try}\ V\ \mathbf{as}\ x\ \mathbf{in}\ M\ \mathbf{otherwise}\ N\ \to_M M\{V/x\}$$

Configuration Equivalence $\boxed{\mathcal{C} \equiv \mathcal{D}}$

$$\mathcal{C} \parallel (\mathcal{D} \parallel \mathcal{E}) \equiv (\mathcal{C} \parallel \mathcal{D}) \parallel \mathcal{E} \qquad \mathcal{C} \parallel \mathcal{D} \equiv \mathcal{D} \parallel \mathcal{C} \qquad (va)(vb)\mathcal{C} \equiv (vb)(va)\mathcal{C}$$

$$\mathcal{C} \parallel (va)\mathcal{D} \equiv (va)(\mathcal{C} \parallel \mathcal{D}), \quad \text{if } a \notin \mathsf{fn}(\mathcal{C})$$

$a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \equiv b(\overrightarrow{W}) \leftrightsquigarrow a(\overrightarrow{V})$ $\qquad$ $() \,||\, \mathcal{C} \equiv \mathcal{C}$

$(va)(vb)(\lightning a \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\varepsilon)) \,||\, \mathcal{C} \equiv \mathcal{C}$ $\qquad$ $(va)(vb)(\lightning a \,||\, a(\varepsilon) \leftrightsquigarrow b(\varepsilon)) \,||\, \mathcal{C} \equiv \mathcal{C}$

Configuration Reduction $\boxed{\mathcal{C} \to \mathcal{D}}$

E-CHANNEL $\mathcal{F}\,[\mathbf{channel}\ @U] \to (va)(vb)(\mathcal{F}\,[(a,b)]) \,||\, a(\varepsilon) \leftrightsquigarrow b(\varepsilon)$, where a, b are fresh

E-FORK $\mathcal{F}\,[\mathbf{fork}\ (\lambda_.M)] \to \mathcal{F}\,[()] \,||\, M$

E-SENDLIN $\mathcal{F}\,[\mathbf{send}_{\text{lin}}\ U\ a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W} \cdot U)$

E-RECEIVELIN $\mathcal{F}\,[\mathbf{receive}_{\text{lin}}\ a] \,||\, a(U \cdot \overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[(U,a)] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W})$

E-SENDUN $\mathcal{F}\,[\mathbf{send}_{\text{un}}\ U\ a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[()] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W} \cdot U)$

E-RECEIVEUN $\mathcal{F}\,[\mathbf{receive}_{\text{un}}\ a] \,||\, a(U \cdot \overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[U] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W})$

E-SENDTYPE $\mathcal{F}\,[\mathbf{sendType}\ @U\ a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W} \cdot @U)$

E-RECEIVETYPE $\mathcal{F}\,[\mathbf{receiveType}\ a] \,||\, a(@U \cdot \overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[(@U,a)] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W})$

E-CLOSE $\mathcal{F}\,[\mathbf{close}\ a] \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[()] \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W} \cdot \text{EOS})$

E-WAIT $(va)(vb)(\mathcal{F}\,[\mathbf{wait}\ a] \,||\, a(\text{EOS}) \leftrightsquigarrow b(\varepsilon)) \to \mathcal{F}\,[()]$

E-SELECTLIN $\mathcal{F}\,[\mathbf{select}_{\text{lin}}\ \ell\ a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W} \cdot \ell)$

E-SELECTUN $\mathcal{F}\,[\mathbf{select}_{\text{un}}\ \ell\ a] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[()] \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W} \cdot \ell)$

E-CASE $\mathcal{F}\,[\mathbf{case}\ a\ \mathbf{of}\ \{m\&\ell_i\ a_i \to M_i\}^{i\in 1..n}] \,||\, a(\ell_j \cdot \overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[M_j] \,||\, a_j(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W})$

E-CANCEL $\mathcal{F}\,[\mathbf{cancel}\ a] \to \mathcal{F}\,[()] \,||\, \lightning a$

E-ZAP $\lightning a \,||\, a(U \cdot \overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W}) \to \lightning a \,||\, \lightning U \,||\, a(\overrightarrow{V}) \leftrightsquigarrow b(\overrightarrow{W})$

E-RECEIVELINZAP $\mathcal{F}\,[\mathbf{receive}_{\text{lin}}\ a] \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[\mathbf{raise}] \,||\, \lightning a \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W})$

E-RECEIVETYPEZAP $\mathcal{F}\,[\mathbf{receiveType}\ a] \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[\mathbf{raise}] \,||\, \lightning a \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W})$

E-WAITZAP $\mathcal{F}\,[\mathbf{wait}\ a] \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[\mathbf{raise}] \,||\, \lightning a \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W})$

E-CASEZAP $\mathcal{F}\,[\mathbf{case}\ a\ \mathbf{of}\ \{\text{lin}\&\ell_i\ a_i \to M_i\}^{i\in 1..n}] \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W}) \to \mathcal{F}\,[\mathbf{raise}] \,||\, \lightning a \,||\, \lightning b \,||\, a(\varepsilon) \leftrightsquigarrow b(\overrightarrow{W})$

E-TRYINRAISE $\mathcal{F}\,[\mathbf{try}\ P\ [\mathbf{raise}]\ \mathbf{as}\ x\ \mathbf{in}\ M\ \mathbf{otherwise}\ N] \to \mathcal{F}\,[N] \,||\, \lightning P$

E-RAISE $P\,[\mathbf{raise}] \to \lightning P$

E-LIFT $\mathcal{G}\,[\mathcal{C}] \to \mathcal{G}\,[\mathcal{D}]$, if $\mathcal{C} \to \mathcal{D}$

4 Some Thoughts about Shared Channel Cancellation

4.1 Counter

- Each side of the buffer keeps a counter for the number of threads with ownership of the endpoint. When one endpoint reaches a counter of 0, the owners of the other endpoint raise an exception when calling receive_.
- When to increase and decrease the counter?
- The counter would decrease after a thread calls cancel on the endpoint. But what if a thread forgets to call cancel? The other side would hang on receive_ because the counter never reaches 0. Also, nothing stops a thread from using the channel again after cancelling, but this shouldn't be a problem if the buffers are cleared at the end of execution. Should all shared channel endpoints in the type context of the thread be cancelled after the thread ends?

- The counter would increase at the moment of the fork. But to know which counters to increase, there needs to be way to check which endpoints are being used in the function provided as argument to fork. And what about the thread that creates the channel? It might or might not use the channel. We could start the counters at 1 on creation, but then the creator thread would have to explicitly cancel both endpoints if it won't use them. This not only adds boilerplate code, but it could also lead to unintended garbage collection of the channel, since both endpoints would reach a counter of 0.
- The counter alone is not enough.

4.2 Register First

- There could be a new BIF to register a thread as a user of the endpoint. This would increment the counter in the buffer for the endpoint. BIF cancel would then decrease the counter.
- But what if it is never explicitly cancelled? The counter would never decrease so the clients would still wait forever on receive_. Also, having to register every thread would lead to more boilerplate code, but Erlang does something similar anyway.
- Calling register more than once on the same endpoint should have no effect, so that it can be called at the start of a recursive function.

4.3 Store Shared Channels

- Each thread could store a list of shared channels that it is registered with, and when the thread terminates for whatever reason, all channels in the list are implicitly cancelled. An explicit cancel would also remove from this list besides decreasing the counter.
- But a thread stuck in a loop never terminates, so the endpoints would still not be cancelled. Unless we can detect threads stuck in a loop, we are stuck with this problem. We could ignore it, but that means we would have to trust that programmers won't mess up...

4.4 Increase Counter on send_ instead of fork

- Not a good idea. The same thread could send twice, but only cancel the endpoint once at the end. On the other hand, if the counter only increases on send_, the counter would not be needed anymore since the client could just check the buffer.

4.5 Is cancelling necessary for shared channels anyway?

- The original reason to add a new BIF cancel was to deal with linear channels: they have to be used exactly once until the end of the protocol, but the protocol would never finish if an exception was raised.
- Shared channels can be used 0 times. The protocol does not need to be cancelled. The problem that the previous ideas tried to solve was clients hanging on receive_ (and pattern matching with unrestricted labels) when no servers are available. A common pattern for this is using timeout and retries (reference needed). This could be an alternate version of receive_ or there could be a new block in the try expression: try receive_ c after T retry x... The after retry block would be executed if receive_ is not successful in T milliseconds for x attempts.

- Erlang has an after block. Ruby might have something similar to after and retry blocks, but this needs to be studied.
- This sounds like the most promising solution. cancel would only work on linear channels. This new syntax inevitably increases code size.